Zen Function Spec

Zen — Function Spec

This document describes the design of **Zen**, the Roku QA / test-automation web console: its architecture, components, HTTP API, data flows, data models, and external integrations.

> **Scope.** This design covers the web application: the **frontend** (React SPA), the **backend** (FastAPI), the **4_runcase** runner it drives, and external integrations. The legacy command-line modules `1_create_test_plan` and `5_activate` are intentionally out of scope.

Contents

1 Contents
2 1. System Overview
  2.1 1.1 Architecture Diagram
  2.2 1.2 Request Flow
3 2. Technology Stack
4 3. Frontend Design
  4.1 3.1 Application shell
  4.2 3.2 Tabs and pages
  4.3 3.3 Tab-visibility configuration
  4.4 3.4 Reusable components
  4.5 3.5 API client
5 4. Backend Design
  5.1 4.1 Application & configuration
  5.2 4.2 Router wiring & path constants
  5.3 4.3 Host / port & frontend configuration
6 5. HTTP API Reference
  6.1 5.1 Device — /v1/device
  6.2 5.2 Profile — /v1/profile
  6.3 5.3 Run Case — /v1/runcase
  6.4 5.4 Roku Test — /v1/rokutest
  6.5 5.5 Branch — /v1/branch/automated_tests
  6.6 5.6 User — /v1/user
  6.7 5.7 TestRail — /v1/testrail
  6.8 5.8 Skill — /v1/skill
  6.9 5.9 Agent — /v1/agent
7 6. AI Assistant Design (RAG)
  7.1 6.1 Components
  7.2 6.2 Chat data flow (use_internal=true)
  7.3 6.3 Search data flow (MCP)
8 7. LLM Interface & Configuration
  8.1 7.1 Endpoints
  8.2 7.2 Authentication
  8.3 7.3 Chat completion request
  8.4 7.4 Temperature handling for reasoning models

8.5 7.5 Chat completion response
8.6 7.6 MCP semantic-search call
8.7 7.7 Error handling & retries
8.8 7.8 Runtime model switching
8.9 7.9 Configuration reference
8.10 7.10 Chat call sequence
9 8. Sequence Diagrams
9.1 8.1 Prompt Chat (RAG)
9.2 8.2 Run Case
9.3 8.3 TestRail Search & Profile Generation
10 9. Test Runner & Data Model (4_runcase)
10.1 9.1 Runner call sequence
10.2 9.2 Profile structure
10.3 9.3 Execution modes
10.4 9.4 Device config model
11 10. TestRail Caching
12 11. External Integrations
13 12. Cross-Cutting Concerns

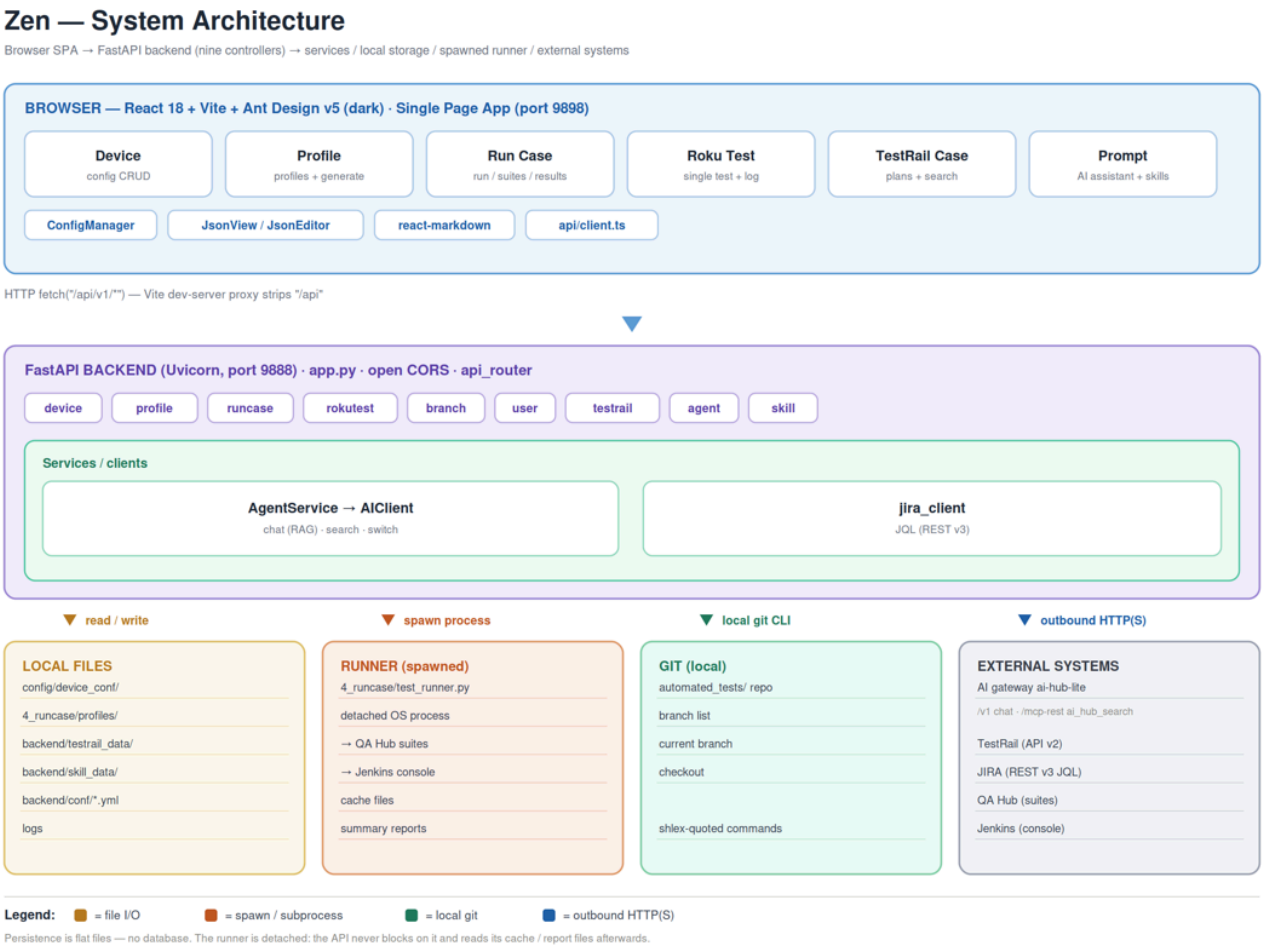
1. System Overview

Zen is a two-tier web app plus a set of external services:

- **Frontend** — React 18 + Vite + Ant Design v5 (dark theme), a single-page app served by the Vite dev server on port **9898**. It proxies `/api/*` to the backend.
- **Backend** — FastAPI (Uvicorn) on port **9888**, exposing `/v1/*` REST endpoints. It reads/writes local config files, drives the `4_runcase` test runner, and calls external services.
- **Runner** — `4_runcase/test_runner.py`, launched as a detached OS process per run; it submits suites to QA Hub and writes cache/report files.
- **External services** — AI gateway (ai-hub-lite), TestRail, JIRA, QA Hub, Jenkins, and local git.

1.1 Architecture Diagram

End-to-end layered view — browser SPA, the FastAPI backend and its nine controllers, the services/clients layer, local storage, spawned runner processes, and external systems.



1.2 Request Flow

How a browser action reaches storage and external systems.

Zen — Request Flow

```
Browser (SPA :9898)
   |  fetch /api/v1/*        (Vite proxy strips /api)
   v
FastAPI backend (:9888) ── local files: device_conf/, profiles/,
testrail_data/, skill_data/, logs
   |                     ── git CLI on automated_tests/
   |                     ── subprocess: 4_runcase/test_runner.py ──>
QA Hub ──> Jenkins
   |                     ── HTTP:
   |                           AI gateway  (ai-hub-lite /v1  + /mcp-
rest ai_hub_search)
   |                           TestRail    (API v2)
   |                           JIRA        (REST v3 JQL)
   |                           QA Hub      (suites)
```

2. Technology Stack

Layer	Technology

Frontend	React 18, TypeScript, Vite 8, Ant Design v5 (`darkAlgorithm`), react-markdown + remark-gfm
Backend	Python 3, FastAPI, Uvicorn, httpx (async), PyYAML, urllib
AI	OpenAI-compatible gateway (ai-hub-lite), MCP tool `ai_hub_search`
Persistence	Flat files: JSON (devices, profiles, TestRail cache), Markdown (skills), YAML (config, testrail plans, users)

3. Frontend Design

3.1 Application shell

`App.tsx` renders an Ant Design `Layout` : a **Header** ("Zen @kz" brand), a left **Sider** menu, and a **Content** card. It is a single-page app with no router — clicking a tab swaps a `page` state value. The theme is dark (`theme.darkAlgorithm` via `ConfigProvider`).

3.2 Tabs and pages

Key	Label	Component	Backend base
`device`	Device	DevicePage → ConfigManager	`/v1/device`
`profile`	Profile	ProfilePage → ConfigManager + generator	`/v1/profile`
`runcase`	Run Case	RunCasePage	`/v1/runcase`
`rokutest`	Roku Test	RokuTestPage	`/v1/rokutest` , `/v1/branch`
`testrail`	TestRail Case	TestRailCasePage	`/v1/testrail`

`agent`	Prompt	AgentPage	`/v1/agent`, `/v1/skill`

3.3 Tab-visibility configuration

Tab visibility is data-driven. `frontend/conf/config.yml` has a `tabs:` map (`device/profile/runcase/rokutest/testrail/agent` → boolean). At build/dev-server start, `vite.config.ts` parses the YAML and injects the map as a JSON string on `import.meta.env.VITE_TAB_VISIBILITY`. `App.tsx` reads it via `loadTabVisibility()`: a tab is shown unless its key is explicitly `false`. The same file also supplies the dev-server `host` / `port` and the backend proxy target.

3.4 Reusable components

Component	Role
`ConfigManager`	Generic CRUD manager for a "resource kind" (Device, Profile). Renders the header toolbar (Refresh / optional extra actions / Add), an optional search box, a Name+Actions table, and the JSON editor modal. Add/Edit/Delete/View are wired to a supplied `CrudApi`.
`JsonView`	Zero-dependency read-only JSON syntax highlighter (keys/strings/numbers/booleans/null colored) rendered into a dark `<pre>`. Exports `jsonModalWidth(text)` which sizes the modal to the widest line (clamped 520–1100px).
`JsonEditor`	Editable JSON field with live syntax highlighting (used for Add/Edit).
`markdown.css`	Styling for rendered Markdown (`.chat-markdown`) used by skills and the assistant answer.

3.5 API client

`api/client.ts` centralizes all HTTP calls through base `/api` (Vite proxies to `127.0.0.1:9888`, stripping `/api`). It surfaces FastAPI `detail` on errors. List helpers strip file suffixes (device/profile `.json`, skill `.md`) so names round-trip cleanly.

4. Backend Design

4.1 Application & configuration

`app.py` builds the FastAPI app via `create_app()`, mounts the aggregate `api_router`, and adds a fully-open `CORSMiddleware` (all origins/methods/headers). It runs under Uvicorn bound to `config.host:config.port` (default `0.0.0.0:9888`).

`conf/config.py` exposes a frozen `Config` dataclass: `host, port, base_url, api_key, key_id, key_alias, model, timeout_sec, max_tokens, temperature`. Each field is resolved by precedence **environment variable → YAML → error**. YAML has two sections — `server_config` (host/port) and `agent_config` (AI gateway settings) — plus an `agent_model` list of selectable models. Environment overrides use the `QATOOLS_*` prefix; `load()` raises `ConfigError` listing all missing keys.

4.2 Router wiring & path constants

`controllers/__init__.py` sets up `sys.path` (adds the QATools root, backend root, and `4_runcase/common`), defines all filesystem path constants, loads the saved TestRail plans, configures the shared logger (console-only), then imports the nine sub-controllers and mounts their routers. Key path constants:

Constant	Location	Holds
`device_conf_dir`	`config/device_conf`	device JSON files
`profile_conf_dir`	`4_runcase/profiles`	profile JSON files
`runcase_dir`	`4_runcase`	`test_runner.py`, cache & report files
`roku_test_dir`	`99_result_history/roku_test`	roku_test logs
`automated_tests_dir`	`automated_tests` (repo)	git branch operations

`testrail_data_dir`	`backend/testrail_data`	cached TestRail case JSON
`testrail_conf_path`	`backend/conf/testrail.yml`	saved plans
`user_conf_path`	`backend/conf/user.yml`	user list
`skill_conf_dir`	`backend/skill_data`	skill folders (`<name>/SKILL.md`)

⚠ **Import ordering.** All module-level names (path constants, `testrail_conf`, `logger`) are defined *before* the sub-controllers are imported, because each controller does `from . import logger, <dir>` at import time. Importing controllers first would raise `ImportError`.

4.3 Host / port & frontend configuration

Zen has two independently-configured processes — the backend (FastAPI, port 9888) and the frontend dev server (Vite, port 9898) — each with its own config file. This subsection consolidates all host/port and tab-visibility settings.

4.3.1 Backend host & port

The backend bind address comes from the `server_config` section of `backend/conf/config.yml`, resolved by `conf/config.py` (env var → YAML → error). Uvicorn binds to `config.host:config.port`.

Setting	YAML (config.yml)	Env override	Default
Backend host	`server_config.host`	`QATOOLS_SERVER_HOST`	`0.0.0.0`
Backend port	`server_config.port`	`QATOOLS_SERVER_PORT`	`9888`

backend/conf/config.yml (excerpt)

```yaml

server_config:
  host: "0.0.0.0"
  port: 9888
```

```
5
```

4.3.2 Frontend host / port & backend proxy target

The frontend is configured entirely by `frontend/conf/config.yml`, read at build/dev-server start by `vite.config.ts`. It sets the dev-server bind (`server`) and the backend the SPA proxies `/api/*` to (`backend`).

Setting	YAML key	Default	Effect
Frontend host	`server.host`	`0.0.0.0`	Vite dev-server bind address.
Frontend port	`server.port`	`9898`	Vite dev-server port.
Backend host	`backend.host`	`127.0.0.1`	Target host for the `/api` proxy.
Backend port	`backend.port`	`9888`	Target port for the `/api` proxy.

frontend/conf/config.yml (excerpt)

```yaml

server:                 # Vite dev server bind
  host: "0.0.0.0"
  port: 9898

backend:                # target of the /api proxy
  host: "127.0.0.1"
  port: 9888

```

`vite.config.ts` parses this file with a small indent-aware reader (no YAML dependency) and applies `server.host` / `server.port` to the dev server and `http://backend.host:backend.port` as the `/api` proxy target (the `/api` prefix is stripped on the way through). Changes take effect on the next dev-server restart.

4.3.3 Left-nav tab visibility

Which tabs appear in the left navigation is controlled by the `tabs:` map in `frontend/conf/config.yml`. Each key maps to a boolean; a tab is shown unless its key is explicitly `false`.

frontend/conf/config.yml — tabs section

```yaml

tabs:
  device:   true
```

```
  profile:  true
  runcase:  true
  rokutest: true
  testrail: true
  agent:    true      # the "Prompt" tab

```

Key	Tab label
`device`	Device
`profile`	Profile
`runcase`	Run Case
`rokutest`	Roku Test
`testrail`	TestRail Case
`agent`	Prompt

Mechanism. At build/dev-server start, `vite.config.ts` reads the `tabs` map and injects it as a JSON string on `import.meta.env.VITE_TAB_VISIBILITY` (via Vite `define`). At runtime `App.tsx` calls `loadTabVisibility()` to parse it: the left menu renders only visible tabs, the initial page is the first visible tab, and hidden pages are never mounted. Setting a key to `false` hides both the menu item and its page. Changes require a dev-server restart (the value is injected at config-load time, not at runtime).

> Example — a read-only deployment that only exposes TestRail browsing and the AI assistant: set `device/profile/runcase/rokutest` to `false`, leave `testrail/agent` as `true`, and restart the frontend.

5. HTTP API Reference

All routes are under `/v1`. 51 endpoints across nine controllers.

5.1 Device — `/v1/device`

Method	Path	Purpose	Params / Body	Returns
GET	`/list`	list device files	—	`{devices: [...]}`

GET	`/{device_name}`	view one device	path name	device JSON
POST	`/add/{device_name}`	create (409 if exists)	body: JSON	`{message}`
PATCH	`/update/{device_name}`	overwrite (must exist)	body: JSON	`{message}`
DELETE	`/delete/{device_name}`	delete	path name	`{message}`

Stored as `<device_conf_dir>/<name>.json`.

5.2 Profile — `/v1/profile`

Method	Path	Purpose	Params / Body	Returns
GET	`/list`	list profiles	—	`{profiles:[...]}`
GET	`/generate_with_testrail_data`	build a profile from a cached TestRail data file	query: `plan`, `device`, `testrail_data_file`	`{message}`
GET	`/{profile_name}`	view one profile	path name	profile JSON
POST	`/add/{profile_name}`	create	body: JSON	`{message}`
PATCH	`/update/{profile_name}`	overwrite	body: JSON	`{message}`

DELETE	`/delete/{profile_name}`	delete	path name	`{message}`

⚠ Route ordering matters: the static `/generate_with_testrail_data` route is declared *before* the dynamic `/{profile_name}` route so it is not shadowed. `plan` is validated as an integer, and `testrail_data_file` is guarded against path traversal.

Profile generation logic. Reads `<testrail_data_dir>/<file>`; template `{name:"<device>_<plan>", description, tasks:[], execution_mode:"SERIAL"}`. For each case that has an `automation_id`, it appends a task whose config sets `case_type` = `BDD` when the id starts with `bdd-` (else `UT`), `case_id` = the id (BDD ids are split on `-`), plus `device`, `testrail_plan_id`, `repeat:0`, and `main` branches. Cases without an automation id are skipped. The result is saved as `profile_<base>_<device>.json`.

5.3 Run Case — `/v1/runcase`

Method	Path	Purpose	Params	Returns
POST	`/run`	spawn `test_runner.py -p <profile> -s <select>` detached	`profile_name`, `select`=All	`{message, pid}`
GET	`/list_running`	running runner processes (ps-parsed)	—	`{running_cases}`
GET	`/status`	same ps-parse	—	`{status}`
POST	`/stop`	`kill -9 <pid>`	`pid`	`{message}`
GET	`/list_cache`	list `*.cache`	—	`{cache_files}`

GET	`/view_cache/{cache_file}`	read cache file	path	`{cache_file_content}`
GET	`/delete_cache/{cache_file}`	delete cache file	path	`{message}`
GET	`/list_report`	list `*summary_report.txt`	—	`{summary_report_files}`
GET	`/view_report/{report_file}`	read report (404 if missing)	path	`{summary_report_content}`
GET	`/delete_report/{report_file}`	delete report	path	`{message}`
GET	`/clear`	remove all cache + report files	—	`{message}`
GET	`/list_suites`	QA Hub suites for a user	`user`, `state`=running	`{suites: [...]}`
GET	`/terminate_suite`	QA Hub terminate	`suite_id`	`{status}`
GET	`/suite_build_url`	resolve Jenkins console URL from suite build number	`suite_id`	`{build_number, url}`

The controller dynamically loads `4_runcase/test_runner.py`. `/run` launches `python3 -u test_runner.py` in a new session with `cwd=runcase_dir`, returning its PID. Running-

process discovery parses `ps aux` for `test_runner.py` and extracts the `-p` / `-s` args.

5.4 Roku Test — `/v1/rokutest`

Method	Path	Purpose	Params	Returns
POST	`/run`	run `roku_test` CLI, log to timestamped file	`device`, `case_id`, `repeat` =0	`{message, command, log_file, pid}`
GET	`/status/{pid}`	check process alive	path pid	`{pid, status, command?}`
GET	`/log/list`	list logs with state + parsed result	—	`{logs: [{name,state,mtime,result}]}`
GET	`/log/view/{log_file}`	read a log (404 if missing)	path	`{log}`
GET	`/log/delete/{log_file}`	delete a log	path	`{message}`

Result parsing scans the log: `res: Error` → Fail, otherwise the last `res: Pass/Fail/Skipped/Blocked`, else Unknown. **State** is "running" when the log name appears in the process table (`ps -eo args | grep roku_test`).

5.5 Branch — `/v1/branch/automated_tests`

Method	Path	Purpose	Body	Returns

GET	`/list`	`git branch`	—	`{automated_tests_branches:[...]}`
GET	`/current`	current branch	—	`{automated_tests_branch}`
POST	`/checkout`	`git checkout` (shlex-quoted)	`{branch_name}`	`{checkout_result}`

Operates local git in `automated_tests_dir`.

5.6 User — `/v1/user`

Method	Path	Purpose	Returns
GET	`/list`	read `user.yml`	`{users:{...}}`

5.7 TestRail — `/v1/testrail`

Method	Path	Purpose	Params / Body	Returns
GET	`/status/list`	live instance statuses	—	`{statuses:[{id,name,label}]}`
GET	`/plan/case/list`	cases of a status under a plan (cached)	`plan`, `status`=Failed, `sync`=false	`{plan,status,count,cases:[...],cached}`

GET	`/plan/list`	saved plans	—	`{plans: [...]}`
POST	`/plan/add`	add a plan, persist YAML	body: plan	`{success}`
DELETE	`/plan/delete`	remove a plan by id	`plan_id`	`{success}`
GET	`/testrail_data/list`	cached files for a plan (by status)	`plan`	`{<status>:"<file>"}`
GET	`/testrail_data/available`	all cached files grouped by plan→status	—	`{plans: {<plan>: {<status>:"<file>"}}}`

Each case record is `{run_id, test_id, case_id, title, automation_id, link}`. Status ids/labels are fetched live (they are instance-specific) rather than hardcoded.

5.8 Skill — `/v1/skill`

Method	Path	Purpose	Body	Returns
GET	`/list`	skill names	—	`{skills: [...]}`
GET	`/{skill_name}`	read SKILL.md	—	`{skill}`
POST	`/add/{skill_name}`	create (400 if exists)	`{skill}`	`{message}`
PUT	`/update/{skill_name}`	overwrite (404 if missing)	`{skill}`	`{message}`

DELETE	`/delete/{skill_name}`	delete	—	`{message}`

Each skill is a folder `<skill_conf_dir>/<name>/SKILL.md` holding Markdown.

5.9 Agent — `/v1/agent`

Method	Path	Purpose	Body	Returns
GET	`/info`	active agent settings	—	`{model, base_url, key_alias, max_tokens, temperature}`
GET	`/models`	selectable models (from config)	—	`[{name, description, model, max_tokens, temperature}]`
POST	`/switch`	change model/params at runtime	`{model?, max_tokens?, temperature?}`	`{message, agent}`
POST	`/chat`	chat, optionally RAG-augmented	`{prompt, use_internal=true, sources?}`	`{response, prompt, model, response_time}`

POST	`/search`	semantic search internal data	`{query, sources?, limit=10}`	`{query, sources, count, results: [...]}`

A singleton `AgentService(AIClient(config))` is built at import; if the config is incomplete, the agent routes return **503**. `/switch` requires at least one of model/max_tokens/temperature and validates ranges (max_tokens > 0, temperature 0–2).

6. AI Assistant Design (RAG)

6.1 Components

- **AgentService** — thin facade: `info / chat / search / switch`.
- **AIClient** — async httpx wrapper over the OpenAI-compatible AI gateway (`ai-hub-lite`, base `/v1`). Auth: `Bearer <api_key>` plus `X-Api-Key-Id` / `X-Api-Key-Alias`. MCP calls go to `<gateway_root>/mcp-rest/tools/call`. A temperature guard omits the `temperature` parameter for reasoning models that only accept `temperature=1` (Claude 4.x/5.x Opus/Sonnet, GPT-5.5/5.6/6, gpt-5-codex).
- **jira_client** — JIRA REST v3 `/search/jql` (v2 removed by Atlassian); Basic auth; returns compacted issue records; also provides a dated JQL cheat-sheet for the model.

6.2 Chat data flow (`use_internal=true`)

Zen answers using a two-path retrieval strategy:

1. **Structured JIRA (JQL) path — tried first.** The model is asked (with a dated cheat-sheet: today's date, last-week Mon–Sun boundaries, `currentUser()` semantics) to translate the question into a single JQL line, or return `NONE`. The output is cleaned and sanity-checked for a JQL operator. If a valid JQL is produced, `jira_client.search_jql()` runs it (up to 25 issues) and the results become the CONTEXT block.
2. **Semantic-search fallback.** If no JQL applies, Zen calls the MCP `ai_hub_search` tool over the selected sources (up to 6 docs, retried up to 3× for transient 5xx). Results are formatted into a numbered CONTEXT block (capped at 12,000 chars). If search fails or is empty, the CONTEXT explicitly instructs the model to say so — and *not* to claim it lacks internal access.
3. Zen always uses the RAG system prompt, sends `CONTEXT (Roku internal data): …` `QUESTION: …`, and returns the completion. With `use_internal=false` it does a plain completion.

```
1
```

```
chat(prompt, use_internal=true)
  ├─ _maybe_jql(prompt) --(model)--> "JQL" or NONE
  │       └─ if JQL: jira_client.search_jql() -> CONTEXT
  └─ else: ai_hub_search (MCP, retry x3) -> CONTEXT
         (empty/fail -> explicit "say so" note)
  -> _complete(RAG_SYSTEM_PROMPT, "CONTEXT ... QUESTION ...")

```

6.3 Search data flow (MCP)

`search()` POSTs to `<gateway_root>/mcp-rest/tools/call` with `{name, server_id:"ai_hub_search", arguments:{query, limit, sources?}}`. The tool name is tried as `ai_hub_search-search` then `search`. Because the gateway intermittently returns 500 "Tool search not found", the whole attempt is retried up to **6×** with linear backoff. The MCP envelope (`content[].text` JSON, or `structuredContent.result`) is unwrapped into the results list; an `isError` envelope raises.

7. LLM Interface & Configuration

All LLM traffic goes through a single async client, `AIClient` (`aiutil/ai_client.py`), which speaks the **OpenAI-compatible** Chat Completions protocol to the Roku AI gateway **ai-hub-lite**. Semantic search uses the gateway's MCP endpoint. This section specifies the wire contract, authentication, parameter handling, error/retry behavior, and the full configuration reference.

7.1 Endpoints

The gateway *base URL* is configured as `base_url` (e.g. `https://ai-hub-lite.msc.rokulabs.net/v1`). Two families of calls are made:

Purpose	Method & URL	Protocol
Chat completion (all model calls)	`POST {base_url}/chat/completions`	OpenAI-compatible
Semantic search (RAG)	`POST {gateway_root}/mcp-rest/tools/call`	MCP tools/call

`gateway_root` is `base_url` with a trailing `/v1` removed — the MCP endpoint lives at the gateway root, outside the versioned API. The client is a single reused `httpx.AsyncClient` created with `base_url`, the auth headers below, and a request `timeout` of `timeout_sec`.

7.2 Authentication

Every request carries these headers (built once at client construction):

Header	Value	Notes
`Authorization`	`Bearer <api_key>`	required
`Content-Type`	`application/json`	required
`X-Api-Key-Id`	`<key_id>`	sent only when configured
`X-Api-Key-Alias`	`<key_alias>`	sent only when configured; identifies the caller (e.g. `kzheng`)

⚠ Secrets (`api_key`, `key_id`) are never returned by any Zen endpoint. `GET /v1/agent/info` exposes only `key_alias`.

7.3 Chat completion request

`_complete(system_prompt, user_content)` builds a two-message conversation and posts it:

POST {base_url}/chat/completions

```
{
  "model": "claude-opus-5-5",
  "messages": [
    { "role": "system", "content": "<system prompt>" },
    { "role": "user",   "content": "<user content>" }
  ],
  "max_tokens": 128000,
  "temperature": 1          // OMITTED for reasoning models (see 7.4)
}
```

Field	Source	Description
`model`	active agent setting	The currently selected model (changeable at runtime via `/v1/agent/switch`).

`messages`	caller	Always exactly one `system` + one `user` message. The system prompt differs by mode: default assistant, RAG assistant, or the JQL-translation prompt.
`max_tokens`	active setting or call override	Output cap. The JQL-translation call overrides this to `200`; normal chat uses the configured value.
`temperature`	active setting	Included only when the model supports a custom temperature (see 7.4).

7.4 Temperature handling for reasoning models

Some models (reasoning models) reject any `temperature` other than `1` and return HTTP 400. The client therefore **omits** the `temperature` field entirely for models whose id starts with a known reasoning-model prefix:

```

_TEMPERATURE_UNSUPPORTED_PREFIXES = (
    "claude-4.", "claude-opus-4-", "gpt-5-codex",
    "claude-5-", "claude-opus-5", "claude-sonnet-5",
    "gpt-5.5", "gpt-5.6", "gpt-6",
)

```

`AgentSettings.supports_temperature` checks the active model against these prefixes; when unsupported, the payload simply has no `temperature` key and the gateway applies the model default. This is why several entries in `agent_model` carry `temperature: 1` — it documents intent, but the value is not actually sent for those models.

7.5 Chat completion response

Responses follow the OpenAI envelope. `_extract_message()` reads `choices[0].message.content`, tolerating either a plain string or a list of content parts

(each with a `text` field), and returns the trimmed text. A non-JSON body, a missing `choices`, or a missing message raises `AIClientError`.

```
{ "choices": [ { "message": { "role": "assistant", "content": "<answer text>" } } ] }
```

7.6 MCP semantic-search call

Search posts an MCP `tools/call` envelope to `{gateway_root}/mcp-rest/tools/call`:

POST {gateway_root}/mcp-rest/tools/call

```
{
  "name": "ai_hub_search-search",   // primary; "search" tried as fallback
  "server_id": "ai_hub_search",
  "arguments": { "query": "<text>", "limit": 10, "sources": ["JIRA"] }
}
```

The response is the MCP envelope: the results arrive as a JSON string inside `content[].text` (or `structuredContent.result`). `_extract_search_results()` unwraps it to a `results[]` list; an `isError:true` envelope raises. Each result is `{document_id, content, ...}`.

7.7 Error handling & retries

Call	Failure	Behavior
Chat completion	transport error / non-2xx / non-JSON	Raise `AIClientError` (surfaced to the controller as HTTP 502). No retry — a bad completion is not transient.
JQL translation	`AIClientError`	Swallowed → treated as "no JQL", falls through to semantic search.

MCP search	500 "Tool search not found" (flaky gateway)	Retry the whole attempt up to **6×** with linear backoff (`0.4 × attempt` s); try tool name `ai_hub_search-search` then `search` each round. A non-"not found" 4xx/5xx breaks early.
RAG search (from chat)	search fails after its own retries	Retried up to **3×** at the chat layer; if still failing, a CONTEXT note tells the model to say it could not reach internal data (never to claim it lacks access).

7.8 Runtime model switching

`POST /v1/agent/switch` mutates the in-memory `AgentSettings` (model / max_tokens / temperature) on the singleton client — no restart, no config-file write. At least one field is required; `max_tokens` must be > 0 and `temperature` in 0–2. The switch is process-local and resets to the configured defaults when the backend restarts. `GET /v1/agent/models` lists the selectable models from the `agent_model` config list.

7.9 Configuration reference

LLM settings come from `backend/conf/config.yml`, resolved by `conf/config.py` with precedence **environment variable → YAML → error**. The YAML path can be overridden with `QATOOLS_AGENT_CONFIG_PATH`. Missing required keys make `load()` raise `ConfigError` listing every missing key (and the agent routes then return 503).

config.yml — `agent_config` section (the active agent):

Key	Env override	Type	Example	Meaning

`base_url`	`QATOOLS_AGENT_BASE_URL`	str	`https://ai-hub-lite.msc.rokulabs.net/v1`	Gateway base (chat at `/chat/completions`; MCP at root).
`api_key`	`QATOOLS_AGENT_API_KEY`	str	*(secret)*	Bearer token.
`key_id`	`QATOOLS_AGENT_KEY_ID`	str	*(secret)*	`X-Api-Key-Id` header.
`key_alias`	`QATOOLS_AGENT_KEY_ALIAS`	str	`kzheng`	`X-Api-Key-Alias` header.
`model`	`QATOOLS_AGENT_MODEL`	str	`claude-opus-5-5`	Default active model at startup.
`timeout_sec`	`QATOOLS_AGENT_TIMEOUT_SEC`	float	`300.0`	Per-request HTTP timeout.
`max_tokens`	`QATOOLS_AGENT_MAX_TOKENS`	int	`128000`	Default output cap.
`temperature`	`QATOOLS_AGENT_TEMPERATURE`	float	`1`	Default sampling temperature (omitted for reasoning models).

config.yml — `server_config` section: `host` (`QATOOLS_SERVER_HOST`, default `0.0.0.0`) and `port` (`QATOOLS_SERVER_PORT`, default `9888`).

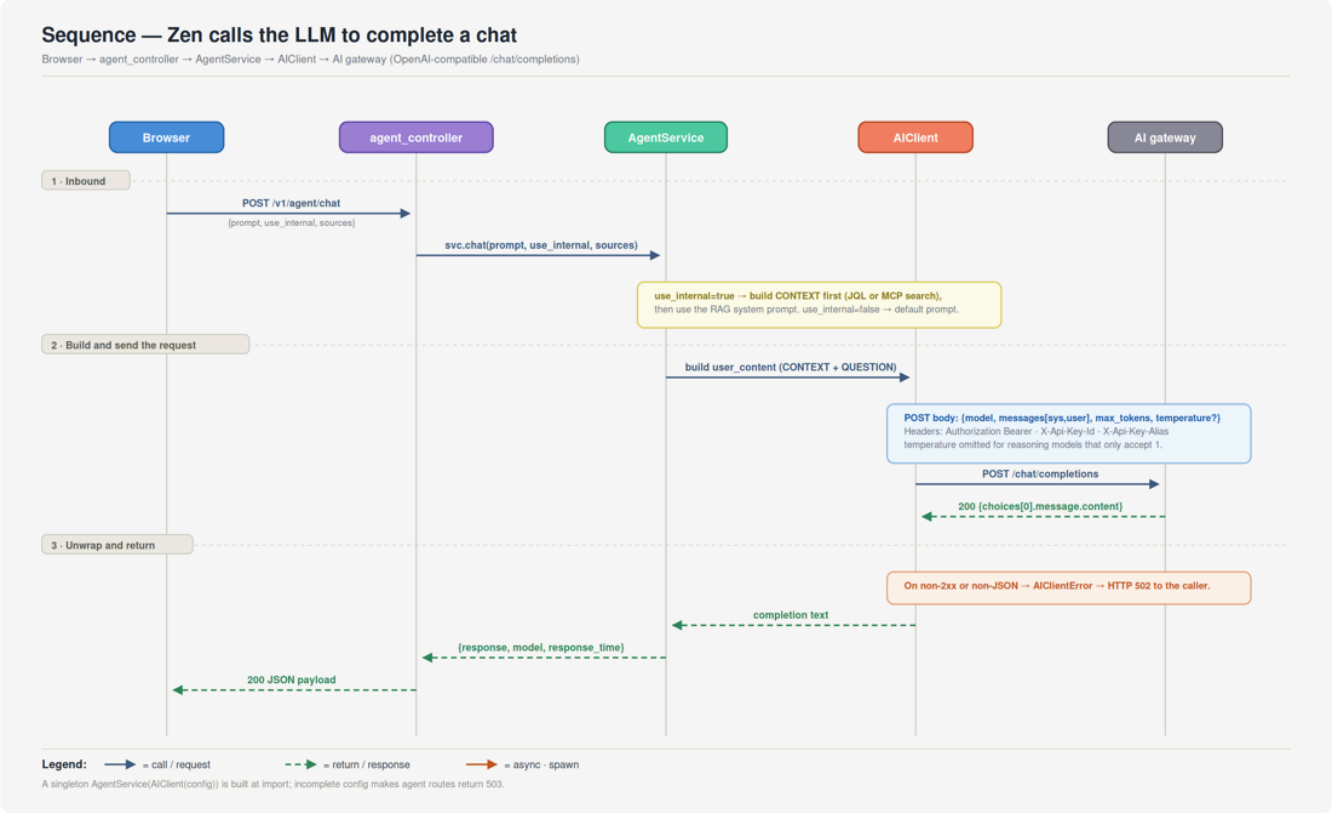
config.yml — `agent_model` list: the models offered by `GET /v1/agent/models` and the Prompt tab's Switch Model dropdown. Each entry is `{name, description, model, max_tokens, temperature}`. Currently configured (8 models):

model	max_tokens	temperature	temperature sent?
`claude-opus-5-5` (default)	128000	1	no (reasoning)
`claude-4.7-opus`	128000	0.7	no (matches `claude-4.`)
`claude-5-sonnet`	128000	0.7	no (matches `claude-5-`)
`gpt-5.4`	128000	0.7	yes
`claude-5-opus`	128000	1	no (reasoning)
`claude-4.6-opus-context-1m`	128000	0.7	no (matches `claude-4.`)
`gpt-5.5`	128000	1	no (reasoning)
`gpt-6-luna`	128000	1	no (reasoning)

To add a model, append an entry to `agent_model` and restart the backend. If the model is a reasoning model that only accepts `temperature=1`, ensure its id matches a prefix in `_TEMPERATURE_UNSUPPORTED_PREFIXES` (or add one) so the client omits `temperature` and avoids a 400.

7.10 Chat call sequence

End-to-end view of a single chat request calling the LLM: from the browser through the controller, service, and `AIClient`, out to the AI gateway's `/chat/completions`, and back. (The RAG retrieval that builds CONTEXT before this call is detailed in §6.2 and §8.1.)

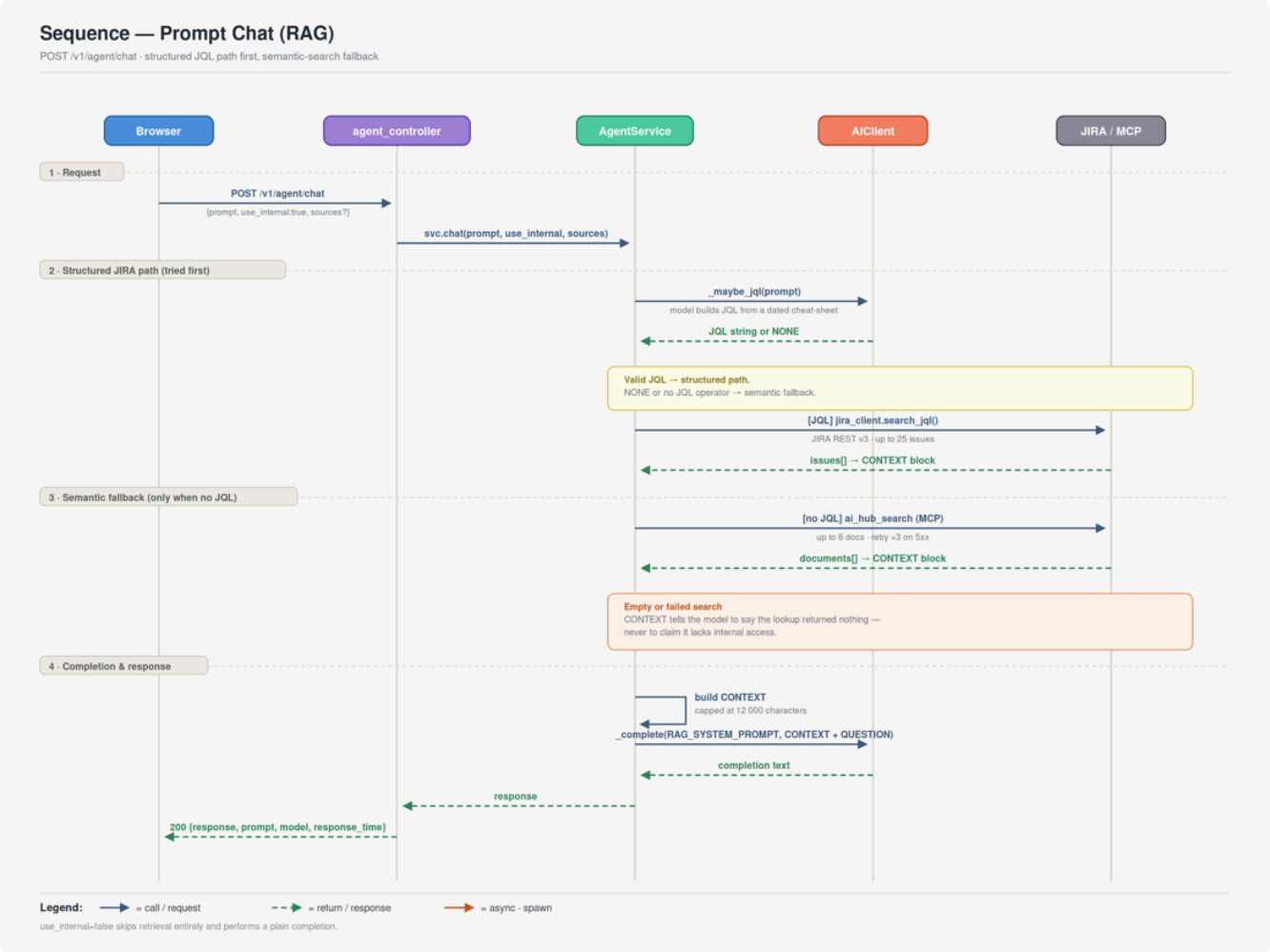


8. Sequence Diagrams

The three most important runtime flows, as message sequences. Solid arrows are calls; dashed green arrows are returns; grey arrows are outbound HTTP to external systems.

8.1 Prompt Chat (RAG)

A chat request with `use_internal=true` follows a two-path retrieval strategy. Zen first asks the model to translate the question into JQL; if a valid JQL is produced it runs it against JIRA (structured path), otherwise it falls back to MCP semantic search. Either way the retrieved documents become a CONTEXT block that is prepended to the question for the final completion.



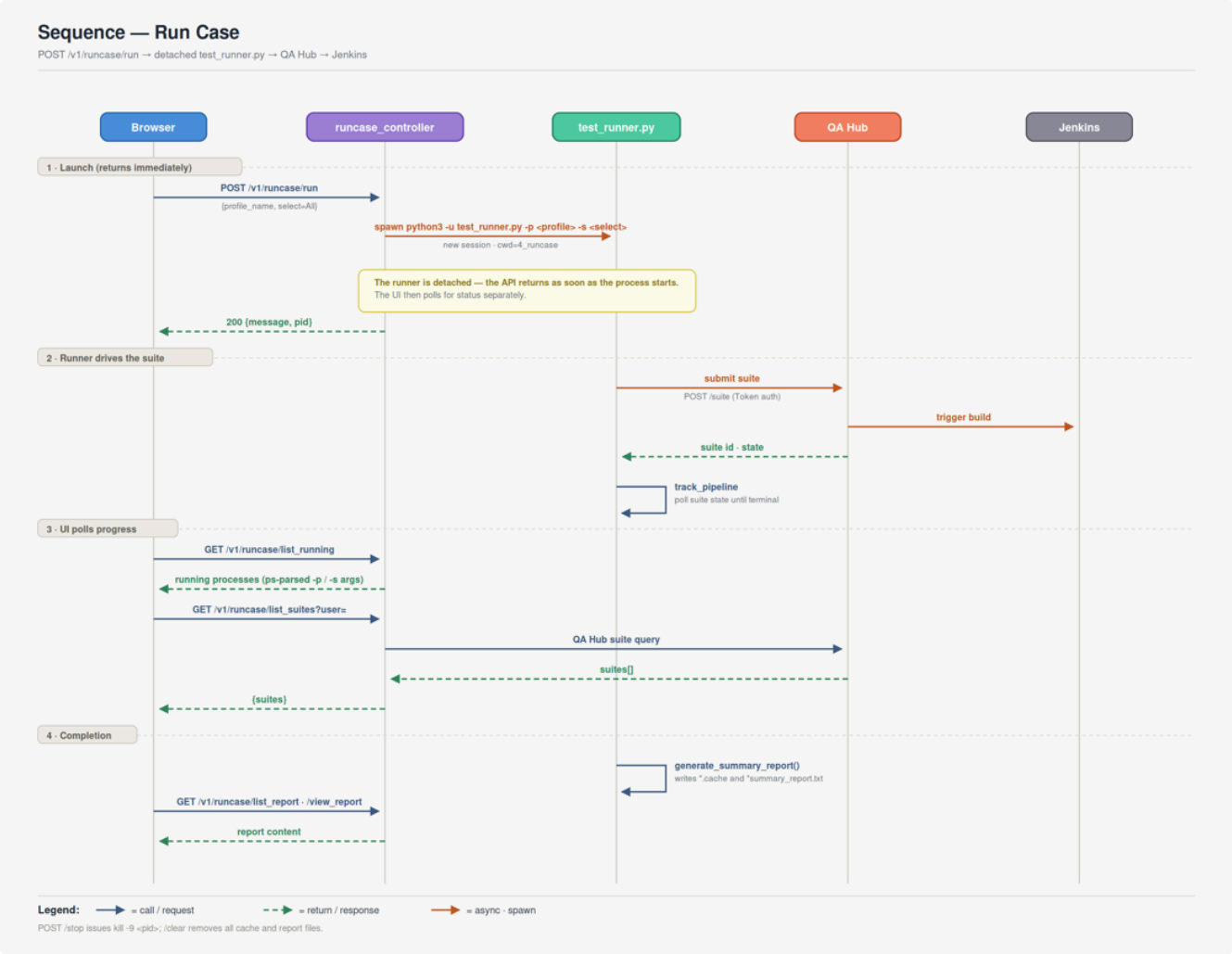
<details>
<summary>Step-by-step</summary>

1. Browser → `agent_controller`: `POST /v1/agent/chat` with the prompt and `use_internal:true`.
2. Controller → `AIClient.chat()`.
3. `AIClient` → AI gateway: `_maybe_jql()` asks the model (with a dated JQL cheat-sheet) to produce a single-line JQL or `NONE`.
4. **JQL path:** if valid, `jira_client.search_jql()` runs it (REST v3) → issues become the CONTEXT block.
5. **Fallback:** otherwise the MCP `ai_hub_search` tool runs (retried up to 6×) → documents become the CONTEXT block.
6. `AIClient` → AI gateway: final `_complete()` with the RAG system prompt + CONTEXT + question.
7. Answer flows back → controller → browser as `{response, model, response_time}`.

</details>

8.2 Run Case

Running a profile spawns a detached OS process; the UI then polls for running processes and QA Hub suites independently. The runner submits a suite to QA Hub, which triggers a Jenkins build, and writes cache + summary-report files locally.

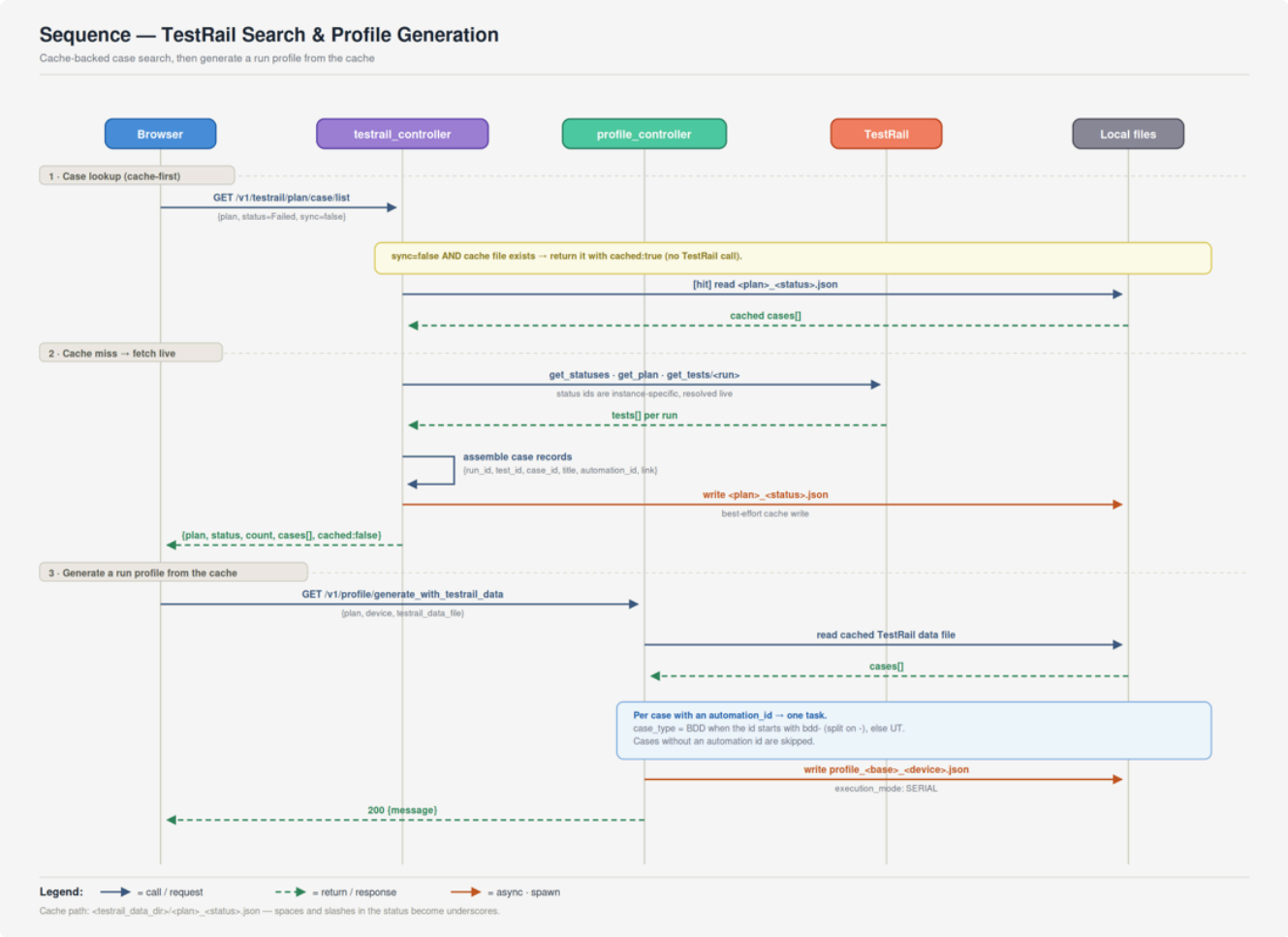


Step-by-step

1. Browser → `runcase_controller`: `POST /v1/runcase/run`. The controller spawns `test_runner.py` in a new session and immediately returns `{message, pid}` — the run is detached.
2. `test_runner.py` → QA Hub: submits a suite; QA Hub triggers a Jenkins build.
3. Browser polls `GET /v1/runcase/list_running` (ps-parsed local processes) and `GET /v1/runcase/list_suites` (QA Hub suites with progress + Jenkins console URL).
4. The runner continuously writes `*.cache` and `*summary_report.txt` files, surfaced under Run Results.

8.3 TestRail Search & Profile Generation

TestRail case lookups are cached as `<plan>_<status>.json`. A subsequent "Add Profile with RunPlan" reads that cache file and builds a runnable profile — connecting the TestRail Case tab to the Profile tab.



<details>
<summary>Step-by-step</summary>

1. Browser → `testrail_controller`: `GET /v1/testrail/plan/case/list`. With `sync=false` and a cache hit, the cached JSON is returned (`cached:true`).
2. **Cache miss / sync:** the controller resolves the status id, enumerates the plan's runs, pulls the tests, and writes `<plan>_<status>.json`.
3. Later, Browser → `profile_controller`: `GET /v1/profile/generate_with_testrail_data` reads that cache file, builds a task per automated case, and writes `profile_<base>_<device>.json`.

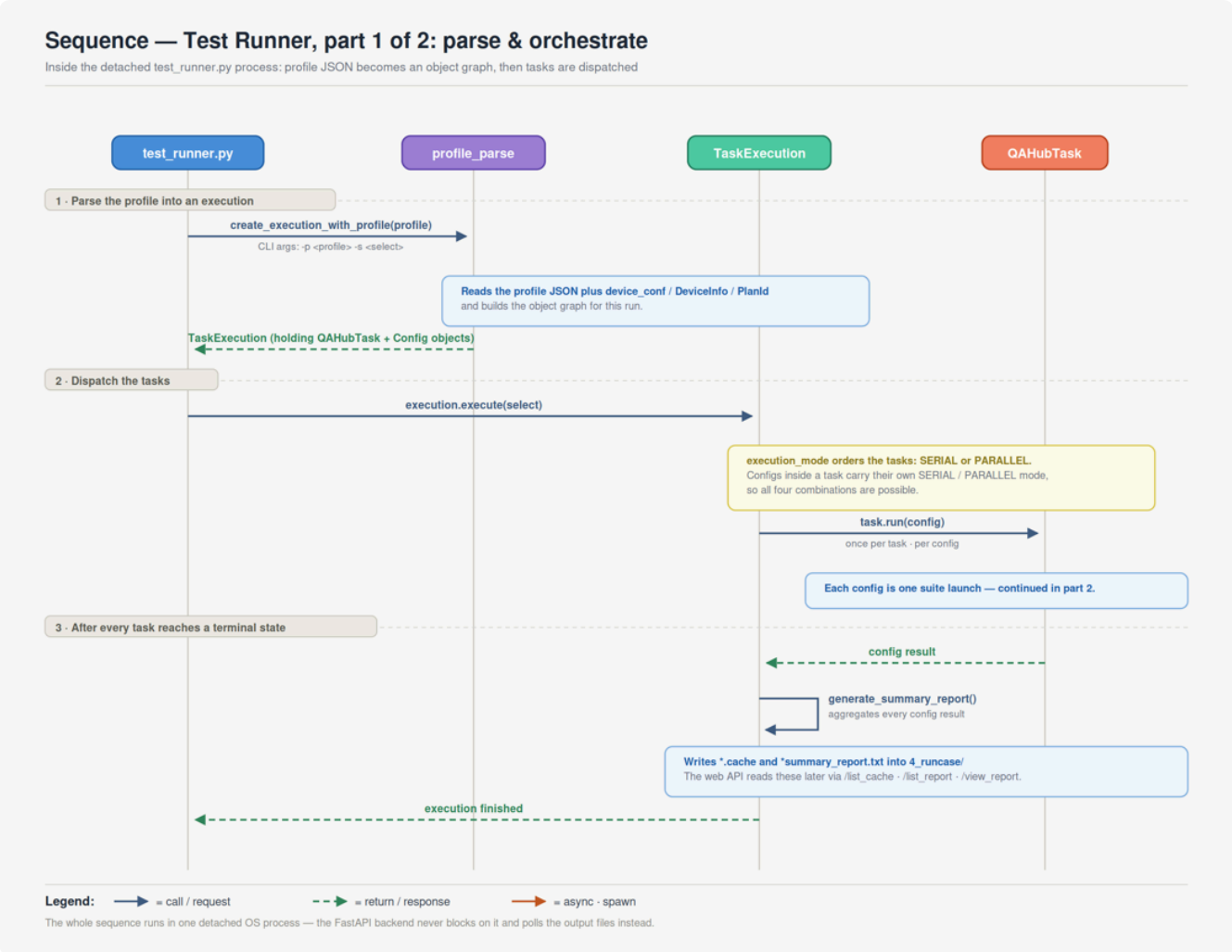
</details>

9. Test Runner & Data Model (4_runcase)

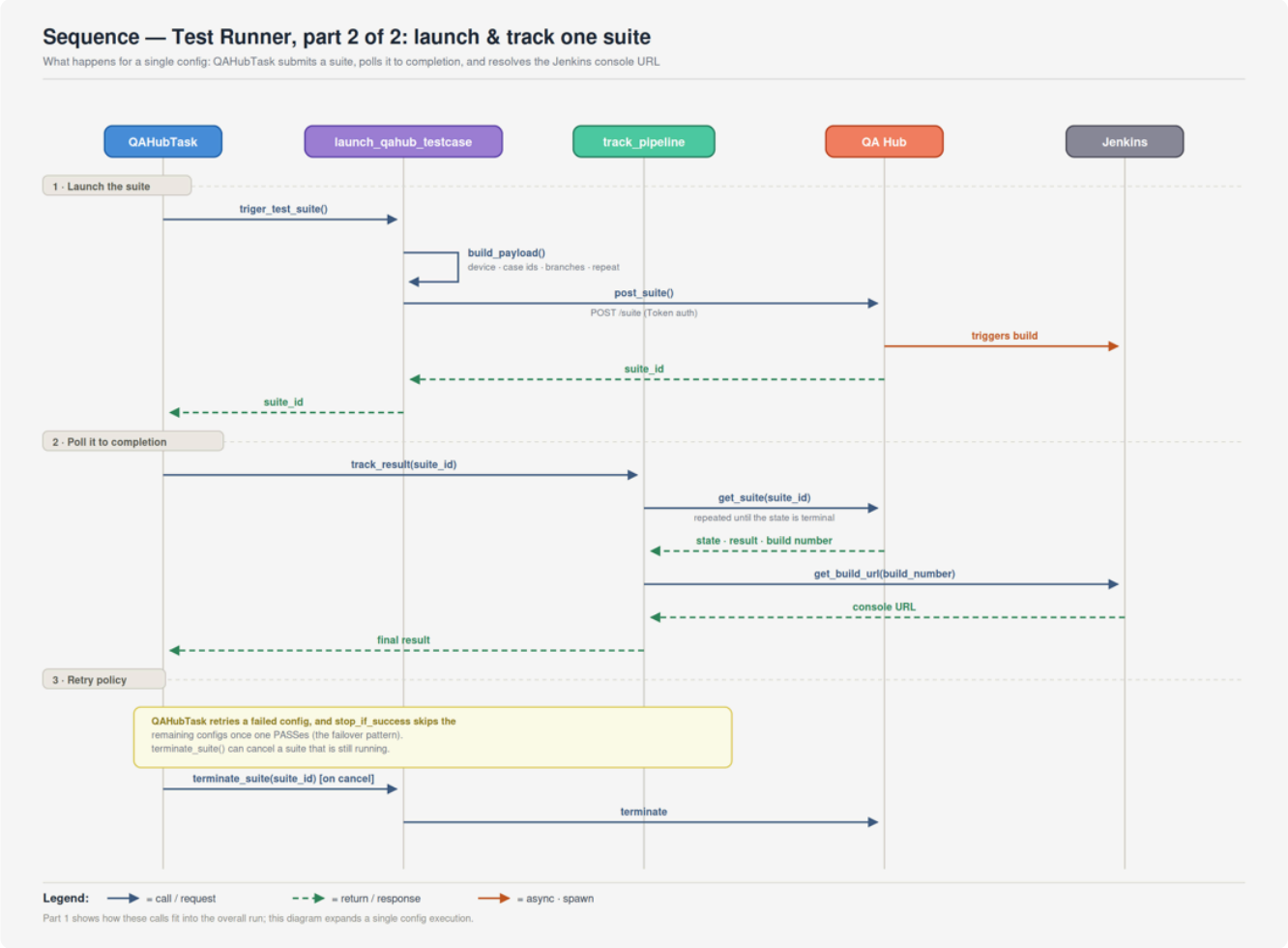
The Run Case and Profile features are built around the `4_runcase` runner and its **profile** data model.

9.1 Runner call sequence

The backend spawns `test_runner.py` as a detached process (POST /v1/runcase/run). The sequences below trace the calls between the runner's modules inside that process. Part 1 covers parsing the profile and dispatching tasks; part 2 expands what happens for a single config — launching a suite on QA Hub and polling it to completion. The table that follows summarises each module's responsibility:



The calls made for each individual config — one suite launch, tracked to completion:



Component	Responsibility
`test_runner.py · Runner`	CLI entry (`-p profile -s select`); calls `create_execution_with_profile()` then `.execute(select)` .
`profile_parse.py`	Reads the profile JSON (plus `device_conf` / `DeviceInfo` / `PlanId`) and builds a `TaskExecution` with its `QAHubTask` / `Config` objects.
`task_handle.py · TaskExecution`	Orders tasks (SERIAL / PARALLEL), runs `execute(select)` , and produces the summary report.

`task_handle.py · QAHubTask`	Runs a task's configs: `triger_test_suite` → `track`, with retry and `stop_if_success`.
`launch_qahub_testcase.py`	`launch_test` → `build_payload` → `post_suite` (POST /suite, Token auth); also `terminate_suite`.
`track_pipeline.py`	`track_result` → `get_suite` (poll state); `get_build_url` resolves the Jenkins console URL.

9.2 Profile structure

```
{
  "name": "my plan",
  "execution_mode": "SERIAL",          // or PARALLEL — order of tasks
  "tasks": [
    {
      "configs": [
        {
          "case_type": "BDD",           // BDD | UT | NEW
          "case_id": "2659886400",      // TestRail id (BDD/NEW) or
Test.method (UT)
          "device": "d_ws22_Sandia_DE", // device config file name (no
.json)
          "testrail_plan_id": 0,         // 0 = no push, or a plan id
          "repeat": 0,                   // extra repeats
          "automation_test_branch": "main",
          "roku_automation_branch": "main"
        }
      ],
      "execution_mode": "SERIAL",       // order of configs within the
task
      "stop_if_success": true,           // stop remaining configs on
first PASS
      "print_console_message": false,
      "timeout": 36000
    }
  ]
}

```

9.3 Execution modes

Tasks	Configs	Behavior

SERIAL	SERIAL	One task at a time; configs one by one. `stop_if_success` skips the rest on first PASS (failover pattern).
SERIAL	PARALLEL	One task at a time; configs within a task run concurrently.
PARALLEL	SERIAL	All tasks concurrent; configs within each run in order.
PARALLEL	PARALLEL	Maximum concurrency.

9.4 Device config model

Device files live in `config/device_conf/`; the file name (minus `.json`) is the device key used by profiles. Fields under a `roku` object:

Field	Required	Description
`ip_address`	Yes	Device IP
`brand_name`	Yes	TV brand
`device_id` / `serial_number`	Yes	Device identifiers
`maxwell_config.host`	No	Tuner simulator IP
`power_config`	No	Power strip: `host`, `outlet`, `model`
`qa_bot_config.host`	No	QA Bot (Raspberry Pi) IP
`rtvsp_config` / `serial_config`	No	IR blaster / serial console

10. TestRail Caching

TestRail lookups are cached to keep the UI responsive and to feed the profile generator.

- **Cache file:** `<testrail_data_dir>/<plan>_<status>.json` (status spaces/slashes → `_`).
- **Read path** (`plan/case/list`): if `sync=false` and the cache file exists, return it with `cached:true`. Otherwise fetch fresh: resolve the `status_id` live, enumerate the plan's run ids (`get_plan` → entries → runs), pull `get_tests/<run>&status_id=<id>` for each, assemble case records with per-test view links, write the cache (best-effort), and return with `cached:false`.
- **Consumers:** `testrail_data/available` and `/list` expose which caches exist; `profile/generate_with_testrail_data` reads a cache file to build a run profile.

11. External Integrations

Service	Endpoint / Base	Auth	Used for
AI gateway (ai-hub-lite)	`https://ai-hub-lite.msc.rokulabs.net/v1` + `/mcp-rest`	Bearer api_key + X-Api-Key-Id/Alias	chat completions; MCP `ai_hub_search`
TestRail	`testrail.eswat.roku.com/testrail` API v2	Basic (email + API key)	plans, runs, tests, statuses
JIRA	`roku.atlassian.net` REST v3	Basic (email + token)	JQL queries for the assistant
QA Hub	`qahub.eswat.rokulabs.net/api/v1`	Token header	suite list / terminate / detail
Jenkins	`jenkins.eswat.rokulabs.net/job/qa_hub_launch_suite`	—	build console URL from suite build number

Git (local)	`automated_tests` repo	local	branch list / current / checkout

⚠️ GitLab appears only as a selectable *source label* for the assistant's semantic search; the backend performs no direct GitLab API calls. Repository operations use the local git CLI.

12. Cross-Cutting Concerns

Concern	Design
CORS	Fully open on the backend (all origins/methods/headers) to simplify the SPA-to-API dev proxy.
Logging	Shared `LogUtil` logger, console-only (file handler removed; stray empty `backend.log` deleted at startup).
Configuration	Backend via `conf/config.py` (env → YAML precedence). Frontend host/port + tab visibility via `frontend/conf/config.yml` injected at build time.
Error handling	Controllers raise `HTTPException` with descriptive detail; the SPA surfaces `detail` as a toast.
Process model	Runs are detached OS processes (Run Case, Roku Test); the UI tracks them by PID and by parsing `ps`.
Secrets	Credentials come from `config/env.py` / environment; never returned by the API (only `key_alias` is surfaced).

Zen Function Design — generated from source at `QATools`*. Excludes* `1_create_test_plan` *and* `5_activate` *per scope.*